

Planificación de procesos y algoritmos de planificación de procesos

Planificación de procesos

Colas de planificación

El sistema operativo presenta una serie de colas que utiliza en la planificación de los procesos de forma que los procesos migrarán entre ellas. Estas colas son: cola de procesos, cola de procesos listos y las colas de dispositivos:

- Cola de procesos: contiene la identificación de todos los procesos en el sistema.
- Cola de procesos listos: está formada por los punteros al primero y al último de los bloques de control de procesos de los procesos que están en estado listo. Cada uno de los demás bloques de la cola tendrá un puntero al siguiente bloque para así conformar la cola.
- Colas de dispositivo: estas colas contienen al conjunto de procesos que están esperando por un dispositivo de E/S, es decir, procesos que están bloqueados o en espera.

Desde su creación hasta su finalización todos los procesos se mueven entre estas diferentes colas de planificación. Como parte de la tarea de planificación se debe seleccionar de alguna manera los procesos que se encuentran en estas colas. Este proceso de selección lo realiza el planificador relacionado con cada una de las colas.

Tipos de planificación. Planificadores

En cualquier clase de sistema, los procesos deben ser asignados al procesador en un orden determinado para que sean ejecutados. El módulo del sistema operativo que implementa el algoritmo que decide que trabajos ejecuta el procesador y en qué orden se llama planificador. Mientras que, usualmente, se entiende por planificación a una asignación concreta de procesos al procesador en intervalos de tiempo específicos. Más allá de esto, es posible afirmar que el objetivo de la planificación de procesos es designar a los procesos a ser ejecutados, de forma que se cumplan los objetivos del sistema tales como el tiempo de respuesta, el rendimiento y la eficiencia del procesador.

Una planificación eficiente de la CPU depende de una propiedad que siempre se presenta en los procesos: la ejecución de un proceso consta de un ciclo de ejecución en la CPU, seguido de una espera de E/S. Todos los procesos alternan entre estos dos estados. Siempre se comienza la ejecución del proceso con una ráfaga de CPU. La misma va seguida de una ráfaga de E/S, a la cual le sigue otra ráfaga de CPU, luego otra ráfaga de E/S y así sucesivamente hasta que, al terminar el proceso, la última ráfaga de CPU concluye con una solicitud al sistema para finalizar la ejecución.

Para la inmensa mayoría de los sistemas operativos la planificación se divide en tres tareas autónomas: la planificación de largo plazo, la planificación de mediano plazo y la planificación de corto plazo. Cada una de estas tareas es realizada por un planificador distinto.

La planificación a largo plazo se realiza cuando se crea un nuevo proceso. En este caso el planificador de largo plazo presenta las siguientes características:

- Es el que decide admitir o no a un nuevo proceso y a qué proceso admitir, para luego ponerlo en la cola de listos.
- Se lo invoca con poca frecuencia, quizás después de transcurridos algunos segundos o minutos. Por ejemplo, se lo puede invocar cuando termina un proceso, o también, si el tiempo que el procesador está ocioso excede un determinado valor. Esta es la razón por la que puede ser lento en su ejecución.
- Controla el grado de multiprogramación determinando qué programas se admiten en el sistema para su procesamiento. Si aumenta el número de procesos creados, disminuirá el porcentaje de tiempo durante el cual cada proceso se pueda ejecutar.
- Debe seleccionar una eficiente mezcla de procesos, equilibrando los procesos limitados por E/S (Procesos que pasan más tiempo utilizando los dispositivos de E/S que el procesador) y los procesos limitados por la CPU (Procesos que realizan mucho trabajo computacional y ocasionalmente usan los dispositivos de E/S).

Si en el sistema operativo no existe un planificador de largo plazo, su tarea la realizará el planificador de corto plazo.

El motivo central que fundamenta la tarea del planificador de mediano plazo es que, en ocasiones, puede ser ventajoso eliminar procesos de la memoria y de esa forma reducir el grado de multiprogramación. Posteriormente, el planificador de mediano plazo puede volver a cargar en memoria central al proceso para así continuar con su ejecución desde el

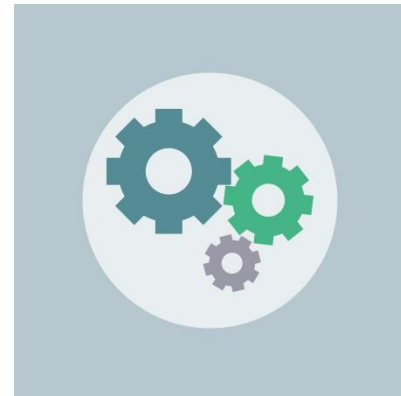
punto en que se la interrumpió. Esto se conoce como intercambio o *swapping* y se lo utiliza para administrar el grado de multiprogramación del sistema

Luego, el planificador de mediano plazo tendrá las siguientes tareas:

- Decidir la carga de un proceso en la memoria central teniendo en cuenta las necesidades de memoria de todos los procesos que están fuera de la misma.
- Descargar y luego volver a cargar el proceso.
- Determinar si realiza o no el intercambio, ya sea para mejorar la mezcla de procesos o porque un cambio en los requisitos de memoria ha hecho que se sobrepase la memoria disponible, requiriendo que se libere memoria.

Por su parte, la planificación de corto plazo impone a su planificador las siguientes funciones:

- Debe decidir cuál de los procesos listos pasará a ser ejecutado. Esta decisión afectará al rendimiento del sistema ya que determinará qué proceso esperará y qué proceso continuará.
- Como se lo ejecuta con mucha frecuencia, por ejemplo, cada vez que un proceso deja al procesador, por una interrupción de reloj, por una interrupción de E/S, por una llamada al sistema o cuando un nuevo proceso pasa al estado listo (dependiendo del algoritmo de planificación), y como el tiempo que hay entre las ejecuciones de los procesos es muy escaso, debe ser extremadamente rápido.



Cambio de contexto

En todo sistema, la conmutación entre procesos permitirá que varios procesos se alternen en el control del procesador. Este cambio requiere que se salve el estado del proceso actual en memoria central y se restaure en el procesador el estado de otro proceso diferente. Esta tarea se conoce como cambio de contexto.

La rapidez con que se realice el cambio de contexto varía de una computadora a otra ya que depende de la velocidad de memoria, del número de registros que tengan que copiarse, de la velocidad del bus y de muchas otras características del hardware. Luego, el tiempo que se emplea en estos cambios de contexto depende sustancialmente del hardware y es tiempo desperdiciado porque que el sistema no realiza ningún trabajo útil.

Despachador

Es el encargado de proporcionar el control de la CPU a los procesos seleccionados por el planificador de corto plazo, lo cual involucra lo siguiente:

- Realizar el cambio de contexto.
- Cambiar al modo usuario.
- Saltar a la posición correcta dentro del programa de usuario para reiniciar dicho programa.

El tiempo que le demanda al despachador realizar todas estas tareas se conoce como latencia de despacho y como esto ocurrirá cada vez que se efectúe una conmutación de procesos, el despachador debe ser lo más rápido posible.

Criterios de planificación

Es posible agrupar los criterios de planificación en dos clases: la de los criterios que están orientados al usuario y la de los criterios que están orientados al sistema. La siguiente es una breve lista de ambas clases:

Criterios orientados al usuario

- Tiempo de permanencia (*turnaround time*): es el tiempo transcurrido desde que se lanza un proceso hasta que finaliza.
- Tiempo de respuesta (*response time*): en un entorno de *time-sharing*, es el tiempo que transcurre desde que se lanza una petición hasta que se comienza a recibir la respuesta.
- Tiempo de espera: es la suma de los períodos de tiempo invertidos en esperar en la cola de procesos listos.
- Fecha límite (*deadlines*): si se puede indicar la fecha límite de un proceso, el planificador debe lograr el mayor porcentaje de fechas límite obtenidas.
- Previsibilidad: se refiere al hecho de que un trabajo dado debería ejecutarse aproximadamente en el mismo tiempo y con el mismo costo a pesar de la carga de procesos que tenga el sistema, ya que, si el tiempo de respuesta o el tiempo de permanencia sufren de una gran variación, la percepción de los usuarios no será positiva.

Criterios orientados al sistema

- *Throughput* o tasa de rendimiento: toda política de planificación debe maximizar el número de procesos completados por unidad de tiempo.
- Uso del procesador: es el porcentaje de tiempo que el procesador está ocupado.
- Equidad: todos los procesos deben ser tratados de la misma manera para que ningún proceso sufra de inanición.
- Prioridades: si se asignan prioridades a los procesos, el planificador siempre debe favorecer a los procesos con prioridades más altas.
- Equilibrado de recursos: el planificador debe mantener ocupados los recursos del sistema favoreciendo a los procesos que utilicen poco aquellos recursos que en un determinado momento sean muy demandados.

Algoritmos de planificación de procesos

Políticas de planificación

Mientras que la función de selección se ocupa de seleccionar a uno de los procesos en estado para darle el uso del procesador, el modo de selección cumple la tarea de establecer el momento en el cuál se ejecutará la función de selección.

La función de selección usualmente se apoya en prioridades o en los recursos necesarios o en cómo será la ejecución del proceso, en cuyo caso, las siguientes variables tienen particular influencia:

- Tiempo total utilizado dentro del sistema hasta este momento.
- Tiempo de uso del procesador hasta este momento.
- Tiempo total de servicio demandado por el proceso. Como aquí se incluye al tiempo de uso del procesador, esta cantidad se deberá estimar o la deberá dar el usuario.

Por su parte el modo de selección se divide en dos clases:

- Sin expulsión, sin desalojo, cooperativo o *nonpreemptive*: una vez que al proceso se le asignó el uso del procesador, es decir está en ejecución, continúa utilizando al procesador hasta que termina o se bloquea para esperar E/S o para solicitar algún servicio al sistema operativo.

- Con expulsión, con desalojo, apropiativo o *preemptive*: un proceso que está usando el procesador (está en ejecución), puede ser interrumpido y pasado al estado de listo por el sistema operativo. La decisión de expulsar puede ser tomada cuando llega un nuevo proceso con mayor prioridad o cuando se produce una interrupción que pasa un proceso de bloqueado a estado de listo o periódicamente por las interrupciones del reloj.

Algoritmos de planificación monoprocesador

Hay muchos algoritmos de planificación de la CPU debido a que existen distintos entornos que requieren algoritmos de planificación diferentes porque lo que el planificador debe optimizar no es lo mismo en todos esos entornos. Reduciendo al mínimo la clasificación, se tendría la siguiente lista:

- Procesamiento por lotes.
- Interactivo.
- De tiempo real.

Planificación en sistemas de procesamiento por lotes

Dentro de esta clase se tienen los siguientes algoritmos de planificación:

Planificación FCFS (*First Come First Served*: primero en llegar, primero en servirse)

También conocido como primero en entrar primero en salir, ya que se administra con una cola FI-FO, o como un sistema de colas estricto es el algoritmo de planificación de la CPU no apropiativo más simple. Se caracteriza por seleccionar los procesos por orden de llegada a la única cola de procesos listos y porque según como lleguen los procesos en la cola de listos será el tiempo medio de espera (tiempo total que estuvo el proceso en estado listo). Dicha cola de procesos listos está ordenada de acuerdo con el instante en que los procesos pasan al estado de listo.

Planificación SJF (*Shortest Job First*: primero el trabajo más corto)

Este algoritmo no apropiativo, cuando la CPU está disponible, se la asigna al proceso que tiene la siguiente ráfaga de CPU más corta. Si las siguientes ráfagas de CPU de dos procesos son iguales usa una planificación FCFS cumplir su tarea. Aunque el algoritmo SJF

es óptimo, porque suministra el tiempo medio de espera mínimo para un conjunto de procesos dado, muchas veces no se lo implementa en el nivel de la planificación de la CPU a corto plazo, ya que como no hay forma de conocer previamente la duración de la siguiente ráfaga de CPU se la estima, teniendo la longitud de la ráfaga de CPU previa y usando una fórmula de promedio exponencial. Otro defecto es que aquí existe el riesgo de inanición para los procesos con las ráfagas más largas. En realidad, este algoritmo pasa a ser un planificador por prioridades, donde la prioridad es la predicción del tiempo de la próxima ráfaga de CPU. Además, como es un algoritmo no apropiativo, una vez que la CPU tiene el proceso, este no puede ser desalojado hasta que completa su ráfaga de CPU.

Planificación SRTF (*Shortest Remaining Time First*: primero el menor tiempo restante)

También llamado SRTN (*Shortest Remaining Time Next*), este algoritmo es una versión apropiativa del algoritmo SJF. Aquí, el planificador siempre selecciona el proceso cuyo tiempo restante de ejecución sea el más corto. Como en el algoritmo SJF, se debe conocer previamente el tiempo de duración de la siguiente ráfaga. Al llegar a la cola de procesos listos un nuevo proceso, el tiempo de su siguiente ráfaga se compara con el tiempo restante del proceso actual. Si el nuevo proceso necesita menos tiempo para ejecutar su próxima ráfaga de CPU que el proceso actual, éste se suspende (pasa al estado listo) y el nuevo proceso toma el control de la CPU. Al igual que en la planificación SJF, existe el riesgo de inanición para los procesos con ráfagas más largas.

Planificación SPN (*Shortest Process Next*: primero el proceso más corto)

Es un algoritmo no apropiativo, similar al SJF, en la que se selecciona el proceso con el tiempo total de procesamiento más corto esperado. De esta forma un proceso corto se situará a la cabeza de la cola, por delante de los procesos más largos. Al igual que en la planificación SJF, en la planificación SPN resulta imprescindible conocer, o al menos de estimar, el tiempo total de procesamiento requerido por cada proceso. Este algoritmo también presenta el riesgo de inanición para los procesos más largos.

Planificación aleatoria

Este algoritmo elige al azar de la cola de procesos listos, al siguiente proceso a ejecutar.

Planificación HRRN (*Highest Response Ratio Next*: primero el de mayor tasa de respuesta)

Al igual que con las planificaciones SRTF, SJF y SPN, este algoritmo necesita calcular la tasa de respuesta de los procesos listo para asignarle el procesador al de mayor tasa de respuesta. A diferencia de las planificaciones antes mencionadas, la planificación HRRN logra que los procesos más largos puedan competir con los procesos más cortos.

Planificación en sistemas interactivos

Esta clase de algoritmos es común en computadoras personales, servidores, etc. Dentro de ella se pueden encontrar los siguientes algoritmos:

Planificación por turno rotativo o circular (*Round Robin*)

Este algoritmo fue diseñado pensando en los sistemas de tiempo compartido. Similar al algoritmo FCFS, pero con la cualidad de desalojar para intercambiar. Esta planificación le otorga una pequeña unidad de tiempo de CPU, denominada quantum de tiempo y usualmente comprendida entre 10 y 100 milisegundos, a cada proceso de la cola de procesos listos. Después de este tiempo el proceso es desalojado y agregado al final de la cola de procesos listos. El planificador de la CPU recorre la cola de procesos listos que es tratada como una cola circular.

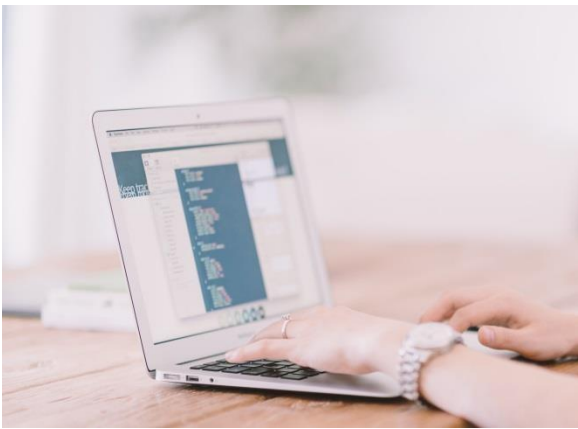
Si hay n procesos en la cola de listos y el tiempo de quantum es q , entonces cada proceso toma $1/n$ del tiempo de CPU en porciones de q unidades de tiempo de una vez como máximo. Ningún proceso espera más que $(n-1) * q$ unidades de tiempo. La performance del algoritmo depende del tamaño del quantum de tiempo. Si el quantum de tiempo es extremadamente largo, la planificación por turnos es igual a la FCFS. Si el quantum de tiempo es muy pequeño (por ejemplo, 1 milisegundo), el método por turnos rotativos se denomina compartición del procesador y, teóricamente, crea la apariencia de que cada uno de los n procesos tiene su propio procesador ejecutándose a $1/n$ de la velocidad del procesador real.

Planificación por prioridades

Es usual en las computadoras con sistemas multiusuario encontrarse con procesos que tienen diferente importancia. Esto hará que a cada uno de dichos procesos se le asigne una prioridad diferente. En ese contexto la planificación presentará las siguientes características:

- Un número de prioridad entero está asociado con cada proceso.
- La CPU es asignada al proceso con la más alta prioridad, en general ocurrirá que el entero más pequeño representa la máxima prioridad.
- Puede ser con desalojo o sin desalojo.
- Puede producir inanición (*starvation*) porque un proceso de baja prioridad podría nunca ser ejecutado. La solución es el *aging* o envejecimiento, por el cual la prioridad del proceso se va incrementando conforme pase el tiempo sin ejecución.

Planificación mediante colas multinivel



Este tipo de planificación se basa en la certeza de poder clasificar a los procesos en diferentes grupos. Por ejemplo, se puede basar la clasificación diferenciando entre procesos interactivos y procesos por lotes.

Puesto que estos dos tipos de procesos tienen requisitos diferentes de tiempo de respuesta y, por tanto, tienen distintas

necesidades de planificación será necesario tener una cola para cada tipo: cola de primer plano o *foreground* para los procesos interactivos y cola de segundo plano o *background* para los procesos por lotes. En tal caso puede ocurrir que los procesos de primer plano tengan prioridad (definida externamente) sobre los procesos de segundo plano y que cada cola tenga su propio algoritmo de planificación, en general Round Robin para la cola de primer plano y FCFS para la otra.

Otra forma de planificación de las colas sería, por ejemplo, con una prioridad fija atendiendo primero a todos los procesos en la cola *foreground* y luego a los de la cola *background*. Como esta planificación puede producir *starvation* deberá considerarse la implementación de alguna técnica de envejecimiento.

También podría aplicarse una planificación por porción de tiempo donde cada cola tiene una cantidad de tiempo del procesador durante el cual puede planificar sus procesos, por ejemplo, el 80% del tiempo de CPU para los procesos de la cola *foreground* con una planificación Round Robin y el 20% restante del tiempo de CPU para los procesos de la cola *background* con una planificación FCFS.

A diferencia de la planificación inmediata anterior, el algoritmo de planificación mediante colas multinivel realimentadas permite el paso de un proceso de una cola a otra.

Una manera de aplicar esta planificación es la de separar a los procesos en función de cómo son sus ráfagas de CPU. Cuando un proceso utiliza demasiado tiempo de CPU, se pasa a una cola de prioridad más baja. Luego, el esquema deja a los procesos limitados por E/S y a los procesos interactivos en las colas de prioridad más alta. Asimismo, un proceso que esté esperando demasiado tiempo en una cola de baja prioridad puede pasarse a una cola de prioridad más alta. De esta forma esta técnica de envejecimiento evita el bloqueo indefinido.

Otra forma de hacerlo es con expulsiones por quantums de tiempo. Cuando un proceso entra en el sistema, se sitúa en la primera cola. Después de su primera expulsión, cuando vuelve al estado de listo, se sitúa en la segunda cola. Es decir que, cada vez que es expulsado, el proceso se sitúa en la siguiente cola de menor prioridad. Así, un proceso corto se completará de forma rápida, sin llegar a migrar muy lejos en la jerarquía de colas de procesos listos. En el otro extremo, un proceso más largo irá degradándose gradualmente. De esta forma, se favorece a los procesos nuevos más cortos sobre los más viejos y largos.

En esta variante puede utilizarse dentro de cada cola una planificación FCFS, excepto en la cola de menor prioridad que utilizará una planificación Round Robin porque el proceso no puede descender más, por lo que es devuelto a esta cola una y otra vez, hasta que se consigue completar. Si, en cambio se decide aplicar una planificación FCFS a todas las colas, el proceso luego de pasar por la cola de menor prioridad, ser ejecutado y expulsado, cuando vuelve al estado de listo, se sitúa nuevamente en la primera cola.

En general, un planificador mediante colas multinivel realimentadas se define mediante los parámetros siguientes:

- El número de colas.
- El algoritmo de planificación de cada cola.
- El método usado para determinar cuándo pasar un proceso a una cola de prioridad más alta.
- El método usado para determinar cuándo pasar un proceso a una cola de prioridad más baja.

- El método usado para determinar en qué cola se introducirá un proceso cuando haya que darle servicio.

Planificación en sistemas de tiempo real

Para interpretar el concepto de sistema de tiempo real resulta conveniente interpretar el concepto de proceso de tiempo real o tarea. Resulta común que un proceso particular trabaje bajo restricciones de tiempo real de naturaleza repetitiva lo cual quiere decir que, el proceso dura un largo tiempo y, durante ese tiempo, realiza cierta función individual (a la que se denominará tarea) en forma repetitiva en respuesta a eventos de tiempo real. De esta forma se puede interpretar que el proceso avanza a través de una secuencia de tareas. En cualquier momento dado, el proceso está dedicando una única tarea y es ese proceso/tarea el que debe ser planificado.

En general, en un sistema de tiempo real, algunas de las tareas son tareas de tiempo real, y éstas tienen cierto grado de urgencia. Tales tareas intentan controlar o reaccionar a eventos que tienen lugar en el mundo exterior. Dado que estos eventos ocurren en tiempo real, una tarea de tiempo real debe ser capaz de mantener el ritmo de aquellos eventos que le conciernen. Así, normalmente es posible asociar un plazo de tiempo límite con una tarea concreta, donde tal plazo especifica el instante de comienzo o de finalización. Esto da lugar a la clasificación de los sistemas de tiempo real en sistemas de tiempo real duro o estricto y de tiempo real suave o blando o no estricto.

En los primeros, una tarea de tiempo real duro será aquella que debe cumplir su plazo límite, es decir que, hay tiempos límites absolutos que se deben cumplir; de otro modo, se producirá un daño inaceptable o error fatal en el sistema. Como ejemplo de este tipo de sistema de tiempo real, se tiene a los sistemas de seguridad crítica como son los sistemas de armamentos, los sistemas antibloqueo de los frenos, los sistemas de gestión de vuelo, los sistemas integrados de salud como los marcapasos. Todos ellos tienen una característica común que los identifica: el sistema de tiempo real debe responder a los eventos con las condiciones de temporización especificadas, caso contrario se podría producir un daño muy grave.

En los segundos, una tarea de tiempo real suave tiene asociado un plazo límite deseable pero no obligatorio; sigue teniendo sentido planificar y completar la tarea incluso cuando su plazo límite ya haya vencido, en otras palabras, no es conveniente fallar en un tiempo límite en ocasiones, pero sin embargo es tolerable. Aquí se tiene como ejemplo a la

inmensa mayoría de los sistemas integrados que no son de seguridad crítica, como es el caso de los conmutadores, enrutadores, relojes inteligentes, *smartphones*, hornos a microondas, *Smart TV*, etc. En todos ellos el no cumplimiento de las condiciones de temporización tendría, como el peor resultado posible, a un usuario descontento.

En ambos casos, el comportamiento en tiempo real se logra dividiendo el programa en varios procesos/tareas, donde el comportamiento de cada uno de éstos es predecible y se conoce de antemano. Por lo general, estos procesos/tareas tienen tiempos de vida cortos y pueden ejecutarse hasta completarse en mucho menos de 1 segundo. Cuando se detecta un evento externo, es responsabilidad del planificador planificar los procesos/tareas de tal forma que se cumpla con todos los tiempos límite.

Por su parte, las tareas que puede llegar a ejecutar un sistema de tiempo real se pueden categorizar como periódicas (que ocurren a intervalos regulares), o como aperiódicas o a plazo fijo (que ocurren de manera impredecible o que ocurren una vez, en un instante determinado). Tal vez un sistema tenga que responder a varios flujos de tareas periódicas. Dependiendo de cuánto tiempo requiera cada tarea para su procesamiento tal vez ni siquiera sea posible manejarlas a todas.

Asimismo, se debe tener en cuenta que un sistema de tiempo real debe limitar el indeterminismo que aparece en los sistemas de tiempo compartido, seleccionando sólo el orden de ejecución de las tareas, que cumplan las condiciones temporales.

Habiendo definido oportunamente el concepto de planificación, se puede decir entonces que la política de planificación es un conjunto de características que incluyen el diseño y la implementación de un sistema de tiempo real. Estas políticas usan dos tipos de planificadores: los planificadores cíclicos y los planificadores por prioridades.

Los planificadores cíclicos establecen de antemano el orden de ejecución de los procesos. Al funcionar el sistema y ejecutarse el planificador, éste se encargará de activar las tareas según el orden definido previamente.

Por su parte los planificadores por prioridades asignan una prioridad a cada tarea en base a su importancia. Al momento de su ejecución, siempre se ejecutará la tarea de mayor prioridad que esté lista en cada momento. Aquí es el planificador quien decide en cada instante qué tarea debe ejecutarse. Esta asignación de prioridades puede ser estática o dinámica.

Será estática cuando la prioridad de cada tarea permanece constante durante toda su existencia dentro del sistema, y entre estos planificadores se destacan el *Rate Monotonic*

(RM) y el *Deadline Monotonic* (DM). Mientras que, será dinámica cuando las prioridades de las tareas varían con el tiempo según unas determinadas reglas, teniendo como ejemplos de planificadores al *Earliest Deadline First* (EDF) y al *Least Laxity First* (LLF). De estas dos clases de planificadores la primera es la más usada.

Finalmente, resulta apropiado enumerar ciertas áreas en las que los sistemas operativos de tiempo real presentan características únicas. Ellas son:

Determinismo

Significa que todo sistema operativo es determinista porque realiza operaciones en instantes de tiempo fijos predeterminados o dentro de intervalos de tiempo predeterminados, pero teniendo en cuenta que, si muchos procesos compiten por recursos y tiempo del procesador, entonces ningún sistema podrá ser totalmente determinista.

Control del usuario

Un sistema de tiempo real debe permitirle al usuario realizar un control de grano fino sobre la prioridad de las tareas para que éste pueda distinguir entre tareas duras y suaves y tenga la capacidad de especificar prioridades relativas dentro de cada clase.

Fiabilidad

Importantísima para los sistemas de tiempo real porque un sistema de este tipo tiene que responder y controlar eventos en tiempo real ya que la pérdida o la degradación de sus prestaciones puede generar consecuencias muy graves.

Reactividad

La reactividad se concentra en establecer cuánto tiempo tarda el sistema operativo, después del reconocimiento, en servir una interrupción.

Operación de fallo suave

Esta característica se refiere a la habilidad del sistema de fallar de forma tal que sea viable preservar tanta capacidad y datos como sea posible.

Evaluación de los algoritmos

En la elección de la política de planificación su rendimiento es una característica fundamental. Aunque se imposible hacer una comparación definitiva, porque el rendimiento relativo dependerá de diversos factores como la distribución de probabilidad de los tiempos de servicio de varios procesos, el tiempo de procesador que se consume debido al propio algoritmo (tiempo gastado en añadir elementos a las colas, ordenar éstas y analizarlas), el mecanismo del cambio de contexto, las demandas de E/S y el funcionamiento de los subsistemas de E/S, la evaluación de los algoritmos de planificación permitirá tener en cuenta los beneficios que presenta cada una de las alternativas a analizadas.

Teniendo en mente lo anterior, un algoritmo menos bueno pero muy simple, como el aleatorio, que consume muy poco tiempo de procesador, puede ser globalmente mejor que otro más sofisticado, puesto que la ganancia de rendimiento conseguida puede no compensar el mayor tiempo consumido. Sin embargo, ¿cómo se selecciona un planificador del procesador para un sistema operativo particular? Ya se han descrito varios algoritmos de planificación, donde cada uno de ellos tiene sus propios parámetros, así que, elegir uno en especial puede resultar extremadamente complicado.

El primer problema radica en definir cuáles son los criterios que se emplearán para seleccionar al algoritmo de planificación. Ya se vieron diferentes criterios de planificación, luego si se quiere seleccionar un algoritmo, primero se tiene que definir la importancia relativa de cada uno de esos criterios. Se pueden incluir muchos diferentes criterios como:

- Maximizar el uso del procesador con la condición de que el tiempo límite de respuesta sea de 1 segundo.
- Maximizar la tasa de procesamiento de forma que el tiempo de ejecución sea linealmente proporcional al tiempo total de ejecución.

Habiendo ya determinado los criterios de selección es posible evaluar entonces los algoritmos que se deseen considerar. Algunos de los diferentes métodos de evaluación son los siguientes:

- Modelado determinista.
- Análisis de colas.
- Modelos de simulación.
- Implementación.

Mientras que los métodos analíticos usan el análisis matemático para determinar el rendimiento de un algoritmo, se tiene que las simulaciones determinan el rendimiento imitando al algoritmo en estudio con una muestra de procesos que sea representativa y calculando el rendimiento resultante. Pero, una simulación sólo puede proporcionar una aproximación al verdadero rendimiento del sistema. Luego, la única metodología cien por ciento confiable para evaluar un algoritmo de planificación es la implementación del algoritmo en un sistema real para controlar su rendimiento dentro de un entorno real.

Planificación multiprocesador

Una de las tantas formas de clasificar a los sistemas con muchos procesadores es la siguiente:

- Débilmente acoplado o multiprocesador distribuido, o clúster: consiste en una colección de sistemas relativamente autónomos; cada procesador tiene su propia memoria principal y canales de E/S.
- Procesadores de funcionalidad especializada: un ejemplo es un procesador de E/S. En este caso, hay un procesador de propósito general maestro y procesadores especializados que son controlados por el procesador maestro y que le proporcionan servicios.
- Procesamiento fuertemente acoplado: consiste en un conjunto de procesadores que comparten la memoria principal y están bajo el control integrado de un único sistema operativo.



De todas ellas, es de la última de la cual se analizarán distintas características relacionadas con la planificación.

- Granularidad: se pueden distinguir cinco categorías de paralelismo que difieren en el grado de granularidad y que están enunciadas en la siguiente lista:
 - Paralelismo independiente.
 - Paralelismo de grano muy grueso.
 - Paralelismo de grano grueso.
 - Paralelismo de grano medio.
 - Paralelismo de grano fino.
- Diseño: todo sistema de multiprocesadores tendrá una planificación que incluirá las siguientes cuestiones interconectadas:
 - Asignación de procesos a procesadores.

- Multiprogramación de cada uno de los procesadores.
- Activación de los procesos.

Al considerar lo anterior, siempre se debe tener en cuenta que la opción elegida dependerá, en general, del grado de granularidad de la aplicación y del número de procesadores disponibles.

- Planificación de procesos: los sistemas de multiprocesamiento tradicionales no vinculan los procesos a los procesadores. Luego, o bien hay una única cola para todo el conjunto de procesadores o, de usarse algún tipo de esquema basado en prioridades, existen múltiples colas basadas en prioridad, proveyendo a ese único conjunto de procesadores. Fuera como fuese, el sistema se puede ver como una arquitectura de colas multiservidor.
- Planificación de hilos: en un sistema multiprocesador los hilos pueden el usar el paralelismo real de una aplicación. Si los hilos de una aplicación se ejecutan simultáneamente en procesadores separados, es posible una remarcable mejora de sus prestaciones. Pero, cuando las aplicaciones necesitan una interacción significativa entre los hilos, como en el paralelismo de grano medio, las pequeñas diferencias en la administración y la planificación de los hilos pueden tener una gran influencia en las prestaciones. Variadas son las propuestas para la planificación de un sistema multiprocesador de hilos y la asignación de los hilos a los distintos procesadores. Entre ellas se pueden encontrar las siguientes:
 - Compartición de carga: los procesos no se asignan a un procesador en particular, sino que se tiene una única cola de hilos listos, y cada procesador, cuando está libre, selecciona un hilo de la cola.
 - Planificación en pandilla: en este esquema el conjunto de hilos interrelacionados se planifica para ser ejecutado simultáneamente en un conjunto de procesadores, en una relación biunívoca.
 - Asignación de procesador dedicado: es lo opuesto a la compartición de carga y provee de una planificación implícita definida por la asignación de hilos a procesadores. En ella, cada proceso ocupa un número de procesadores igual al número de hilos del proceso, durante toda la ejecución del programa. Cuando el programa termina, todos los procesadores vuelven al conjunto para su posible asignación a otro proceso.
 - Planificación dinámica: en esta propuesta, el número de hilos de un proceso puede cambiar durante su ejecución, lo cual permitiría que el sistema operativo ajusta la carga de trabajos para mejorar la utilización de los procesadores.

Bibliografía consultada

- Carretero Pérez, J., De Miguel Anasagasti, P., García Carballeira, F. y Pérez Costoya, F. (2001). *Sistemas Operativos: Una visión aplicada* (2ª ed.). Buenos Aires: Mc Graw Hill.
- Silberschatz, A. y Galvin, P. B. (2006). *Fundamentos de Sistemas Operativos* (7ª ed.). México: Addison Wesley Logman.
- Stallings, W. (2005). *Sistemas operativos: Principios de diseño e interioridades* (5ª ed.). Madrid: Pearson/Prentice Hall.
- Tanenbaum, A. S. y Van Steen, M. (2009). *Sistemas operativos modernos* (3ª ed.). México: Pearson/Prentice Hall.